Ders 16 Genişletilmiş Notu: Vize Çözümleri, HashMap, Java Collection Framework ve İstisna Yönetimi

Ders 16 Genişletilmiş Notu: Vize Çözümleri, HashMap, Java Collection Framework ve İstisna Yönetimi

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Vize Sorularının Çözümü
- 2. Map ve Hash Veri Yapısı
- 3. HashMap Kullanımı
- 4. Java Collection Framework
- 5. ArrayList ve Iterator
- 6. LinkedList ve ArrayList ile Karşılaştırması
- 7. Değeri Liste Olan HashMap: Basit Bir Veritabanı
- 8. İstisna Yönetimine Giriş
- 9. try, catch ve finally
- 10. Kendi İstisna Sınıfımızı Tanımlamak
- 11. Hata Sonrası Akış, Çoklu catch ve İç İçe try
- 12. throw ve throws
- 13. final, finally ve finalize
- 14. Override Edilen Metotlarda İstisna Kuralları
- 15. Özel İstisna Sınıfı Örneği ve System.err

1. Vize Sorularının Çözümü

Hoca derse sınav sorularının çözümüyle başladı. Soruların konusu aynıydı: UML'de verilen sınıfları Java'ya çevirmek, bir metodun sekans diyagramını çizmek ve bu sınıfları `main` içinde kullanmak. Hocaya göre en zorlanılan yerler 5. sorudaki sekans diyagramı ile 4. sorudaki `Actor` sınıfının bir kısmıydı. Aşağıdaki çözüm, kısa nottaki cevap anahtarından alınmıştır. Sınıflar tek tek anlatıldıktan sonra tam kod verilmiştir.

1.1 Person Sınıfı ve UML'e Sadık Kalmak

Birinci soruda `Person` sınıfı isteniyordu. UML'de soyut (abstract) olarak verilmişti. İçinde bir constructor ve `getName` metodu vardır.

Bazı çözümlere UML'de olmayan bir `toString` metodu eklenmişti. Hoca bunu doğru bulmadı ve gerekçesini gizli bilgi (hidden information) kavramıyla açıkladı: UML'de bir bilgi gizlenecekse bu üye alanlar üzerinde olur. Metot gizlenerek bilgi gizleme yapılmaz, gizli metot diye bir şey söylenmedi. Dolayısıyla UML'de görünmeyen bir metodu "gizlenmiş" sayıp eklemek doğru değildir.

> **Hocanın vurgusu:** UML'in dışına çıkılmaması gerekiyor. Gizli bilgi varsa o bir üye alandır, metot değildir.

Soru: Bu `Person` içinde soyut olan ne olacak? Hiç soyut metot yoksa hata vermiyor mu?

Cevap: Bir sınıf soyut olursa soyut metot içermek zorunda değildir. Ama soyut metot içeriyorsa sınıfın soyut olması zorunludur. Öğrenci IntelliJ'de sürekli hata aldığını söyleyince hoca, kodda başka bir yanlış olduğunu düşündüğünü, kodun gözden geçirilmesi gerektiğini söyledi.

Bu kural javac ile denendi. Soyut metodu olmayan `abstract class` hatasız derlenir. Soyut sınıf olmayan bir sınıfa soyut metot yazılınca `B is not abstract and does not override abstract method m() in B` hatası alınır.

> **Hocanın vurgusu:** Bir sınıf soyutsa soyut metot içermek zorunda değildir, ama soyut metot içeriyorsa soyut olmak zorundadır.

1.2 Act Arayüzü

İkinci soruda üç metotlu bir `Act` arayüzü (interface) isteniyordu: `getName`, `getYear`, `getMoreInfo`. Bazı öğrenciler arayüzün içine kod yazmaya çalışmıştı. Hoca arayüzün içine metot yazılmadığını hatırlattı.

> **Ders sonrası not:** Hoca, arayüz içine metot yazılmayacağını söyledi; bu sorudaki düz metotlar için doğrudur. Gövdeli düz metot javac ile denendi ve `interface abstract methods cannot have body` hatası verdi. Ancak `default` ve `static` işaretli metotlar arayüzde gövdeyle yazılabilir; bu da javac ile denendi ve derlendi. Bu sınavın çözümünde bunlara gerek yoktur.

1.3 Movie Sınıfı

Üçüncü soruda `Movie` sınıfı isteniyordu. Bu sınıf `Act` arayüzünü implement eder. Constructor'ı, IMDb puanı için get ve set metotları, `getName` ve `getMoreInfo` metotları vardır. Burada da aynı uyarı geçerlidir: UML'de olmadığı için `toString` eklenmez.

1.4 Actor Sınıfı

Hocanın en zor soru olarak andığı `Actor` sınıfıydı. Hoca bunun derste kitaplığa kitap ekleme gibi örneklerle benzerinin yapıldığını hatırlattı. Sınıfta bir sahne adı (`stageName`) ve referans tutan bir dizi vardır. Bu dizinin elemanları `Act` türündedir, yani bir arayüz türünde referans tutar.

Bu mümkün müdür? Evet. Arayüz türünde nesne üretilemez, ama arayüz türünde referans tutulabilir. Diziye `Act`'i implement eden `Movie` nesneleri eklenir.

Hoca çözüm üzerindeki şu ayrıntılara değindi:

- Üst sınır sabitinin (`maxAct`) çözümdeki gibi yazılması zorunlu değildir. `static` olması da gerekmez. Bir sayaç tutmak gerekir, ama başka biçimde yazılan sayaçlar da kabul edilebilir.
- Dizi nesneye aittir, sınıfa ait değildir. Bu yüzden dizi, nesne oluşturulurken oluşturulmalıdır. Hoca, dizinin yanlış yerde `new` ile oluşturulmasını doğru bulmadı.
- Alt sınıfın constructor'ında `super` çağrısı yazılmalıdır.

  > **Ders sonrası not:** `Person` sınıfında parametresiz constructor olmadığı için `Actor` constructor'ında `super(name)` yazılmazsa kod hiç derlenmez. Aynı durum javac ile denendi ve `constructor P in class P cannot be applied to given types` hatası alındı. Yani bu yalnız bir üslup konusu değil, derleme zorunluluğudur.

1.5 addAct: İsimle Değil, Nesneyle Karşılaştırmak

Hoca en çok `addAct` metodunda sorun gördüğünü söyledi. Önce eklenecek nesnenin daha önce eklenip eklenmediğine bakmak gerekir. Bunun için bir `for` döngüsüyle dizinin başından eleman sayısına kadar tek tek kontrol yapılır. Aynı nesne varsa `return` ile çıkılır, yoksa eleman ilgili yere eklenir.

Bu kontrol **isim üzerinden değil, nesne üzerinden** yapılmalıdır. Hoca gerekçesini şöyle kurdu: İsim nesneyi temsil etmez. Ahmet adında iki kişi olabilir. İsim ve soyisim birlikte kullanılsa bile sorun değişmez.

> **Ders sonrası not:** Hoca "isim primitif bir veridir" dedi; Java'da `String` ilkel tür değil, bir sınıftır. Hocanın asıl söylemek istediği, tek bir alanın nesnenin kimliğini temsil etmediğidir. Bu fikir doğrudur.

Soru: Aynı isim de olsa derken, zaten yalnız isim tutulmuyor mu?

Cevap: Evet, aynı isimler olabilir ve başka bir bilgi de yok. Bu yüzden karşılaştırma isimler üzerinden değil, nesneler üzerinden yapılmalıdır. Nesneler doğrudan == ile karşılaştırılabilir, çünkü `main`'de önceden oluşturulmuş `Film 1`, `Film 2` gibi nesneler metoda referanslarıyla gelir. == işlemi de bu referansları karşılaştırır. İçindeki bilgiler karşılaştırılacaksa == kullanılmaz; o zaman alanlara tek tek bakılır. Birden çok alan olsaydı hepsini tek tek karşılaştırmak gerekirdi. Bu da iyi bir çözüm değildir, çünkü burada benzersiz olan nesnenin kendisidir.

> **Hocanın vurgusu:** Benzersizliği isimle değil, nesneyle kontrol edin. Yapılan isim karşılaştırması size yalnız "bu isimde biri var mı?" sorusunun cevabını verir, "bu nesne eklenmiş mi?" sorusunun değil.

> **Ek bilgi (derste anlatılmadı):** == iki referansın aynı nesneyi gösterip göstermediğine bakar. İki ayrı `String` nesnesinin metni aynı olsa bile == sonucu `false`, `equals` sonucu `true` çıkar; `new String("Ahmet")` ile yapılan deneme javac ile denendi ve bu sonucu verdi. Bu yüzden metin içeriği `equals` ile karşılaştırılır.

Hoca bir de algoritma hatasına dikkat çekti. Bazı çözümlerde diziye eleman eklemek için iç içe iki `for` döngüsü kullanılmıştı. Bir diziye eleman eklerken diziyi iç içe iki kez gezmek gereksizdir ve kötü bir algoritmadır.

> **Hocanın vurgusu:** Nesneye yönelik programlama yaptığımız için algoritma bilgisini bir kenara koymayın. Çözümler algoritmik açıdan da değerlendirilir.

1.6 toString Metodu

`Actor` sınıfındaki `toString` metodu genel olarak doğru yazılmıştı. Bazı çözümlerde her bir yapımın `getName`, `getYear` ve `getMoreInfo` bilgilerini yazmak için ek bir `private` yardımcı metot eklenmişti. Hoca bunu kabul edilebilir buldu, ama genel kuralı yineledi: Mümkün oldukça UML'i ihlal etmemeye çalışın.

1.7 Sekans Diyagramı

Beşinci soru, `toString` metodunun sekans diyagramıydı. Diyagram şu soruyu cevaplar: Bu metot nasıl çalışır? Akış şöyledir:

1. Metodu kimin çağırdığı önemli değildir. Bu yüzden çağıran taraf boş bırakılır. İstek `Actor` sınıfına gider.
2. `Actor` içinde önce `getName` yapılır ve sonuç `info` değişkenine yazılır. `getName` aslında `Person`'dan gelse de sınıfın içindeymiş gibi düşünülür. Üst sınıfa gitmek gerekmez.
3. Sabit metinler eklenir, ardından sahne adı alınır ve `info`'ya eklenir. Kodda doğrudan alan olarak yazılmış olsa da diyagramda `getStageName` olarak gösterilir.
4. `Actor`'dan `Act` tarafına `getName` isteği gider, cevabı geri döner.
5. `getYear` isteği gider, cevabı geri döner.
6. `getMoreInfo` isteği gider, cevabı geri döner. Kodda 4-6. adımlar, eklenen her yapım için `for` içinde yinelenir.
7. İşlem bitince `toString`'i çağıran yere, isteğin cevabı (reply) olarak `info` döndürülür.

Çizimlerdeki yaygın hata, `info`'nun hemen oluşmuş gibi gösterilmesiydi. Oysa arada `get` metotlarının çağrıldığı birçok iş vardır ve bunlar diyagramda görünmelidir.

1.8 main ve Tam Çözüm

Son soruda `main` içinde iki üç nesne üretilip `toString` çağrılıyordu. Hoca, `toString`'i açıkça yazmadan yalnız nesneyi yazdırmayı da kabul etti, çünkü `System.out.println(keanu)` zaten

`toString`'i çağırır.

Aşağıdaki kod kısa nottaki cevap anahtarıdır. Sonuna, derste anlatılan biçimde bir `main` eklenmiştir. Film yılları, sahne adı ve IMDb değeri altyazıda okunmadığı için temsilîdir. Kod javac ile derlenip çalıştırıldı.

```
public abstract class Person {
    private String name;
    public Person(String name) { this.name = name; }
    public String getName() { return name; }
}

public interface Act {
    public String getName();
    public int getYear();
    public String getMoreInfo();
}

public class Movie implements Act {
    private String movieName;
    private int year;
    private double imdbScore;

    public Movie(String name, int year) {
        this.movieName = name;
        this.year = year;
    }
    public double getImdbScore() { return imdbScore; }
    public void setImdbScore(double score) { this.imdbScore = score; }

    public String getName() { return movieName; }
    public int getYear() { return year; }
    public String getMoreInfo() { return "(imdb: " + imdbScore + ")"; }
}

public class Actor extends Person {
    private String stageName;
    private Act acts[];
    private final static int maxAct = 50;
    private int actCount = 0;

    public Actor(String name, String stageName) {
        super(name);
        this.stageName = stageName;
        acts = new Act[maxAct];    // nesneye ait dizi
    }
    public String getStageName() { return stageName; }
    public void setStageName(String stageName) {
        this.stageName = stageName;
    }
    public void addAct(Act anAct) {
        for (int i = 0; i < actCount; i++)
            if (anAct == acts[i])  // referans karşılaştırması
                return;
        if (actCount < maxAct) {
            acts[actCount] = anAct;
            actCount++;
        }
    }
    public String toString() {
        String info;
```

```
        info = "My name is " + getName();
        info += " and my stage name is " + stageName + ". ";
        info += "My acts are: ";
        for (int i = 0; i < actCount; i++)
            info += " " + acts[i].getName() + "(" + acts[i].getYear()
                    + ")" + acts[i].getMoreInfo();
        return info;
    }
}

public class TestActor {
    public static void main(String[] args) {
        Actor keanu = new Actor("Keanu", "Sahne Adi");
        Movie film1 = new Movie("Film 1", 2000);
        Movie film2 = new Movie("Film 2", 2001);
        keanu.addAct(film1);
        keanu.addAct(film2);
        keanu.addAct(film1);        // aynı referans: eklenmez
        System.out.println(keanu);  // toString otomatik çağrılır
    }
}
```

Çıktı:

```
My name is Keanu and my stage name is Sahne Adi. My acts are:
 Film 1(2000)(imdb: 0.0) Film 2(2001)(imdb: 0.0)
```

Çıktı tek satırdır, sayfaya sığması için burada bölündü. Üçüncü `addAct` çağrısı aynı referansı gönderdiği için `film1` ikinci kez eklenmedi.

2. Map ve Hash Veri Yapısı

Sınav çözümünden sonra hoca, geçen haftaki koleksiyon konusundan kalan **Map** kısmına geçti. Dersin planını şöyle özetledi: Önce slayttaki map kısmı, sonra ders notlarında olmayan ama önemli gördüğü koleksiyon ayrıntıları, en sonda istisnalar.

Öğrencilere veri yapıları dersinde hash görüp görmediklerini sordu. Henüz görülmediği için kısaca anlattı.

2.1 Hash Fikri

Amaç, dizideki bir elemana sabit zamanda erişmektir. Hash veri yapısı bir hash fonksiyonu kullanarak elemanın yerini doğrudan hesaplar. Bulma, silme ve oluşturma işlemleri bu sayede hızlı yapılır. Hoca Big-O cinsinden bunun $O(1)$'e kadar inebildiğini söyledi.

> **Ders sonrası not:** Hoca slaytı okurken "non-polynomial time, in constant time, like O1 or O2" dedi; kastettiği, cümlenin devamındaki sabit zamandır, "non-polynomial" ifadesi büyük olasılıkla dil sürçmesidir. $O(2)$ de büyük-O tanımı gereği $O(1)$ ile aynı sınıftır, ayrı bir sınıf değildir.

2.2 Sayısal Örnek: Mod ile Yerleştirme

Boyutu 7 olan bir tamsayı dizisi alınır. Hash fonksiyonu olarak mod işlemi kullanılır, modun tabanı dizi boyutudur:

$$\text{konum} = \text{anahtar} \bmod 7$$

Eklemeler adım adım şöyledir. Bütün değerler python ile hesaplandı.

Eklenen	mod 7	Bakılan konumlar	Yerleştiği konum	Açıklama
17	3	3	3	Doğrudan yerleşir
18	4	4	4	Doğrudan yerleşir
5	5	5	5	Doğrudan yerleşir
12	5	5, 6	6	5 dolu, çakışma var, sonraki yere konur
13	6	6, 0	0	6 dolu, dizinin sonuna gelindi, başa dönülür

Son durumda dizi şöyledir:

```
konum:   0    1    2    3    4    5    6
değer:  13    -    -   17   18    5   12
```

Arama de aynı mantıkla yapılır. 5'i aramak için $5 \bmod 7 = 5$ hesaplanır ve eleman doğrudan bulunur.

Çakışma (collision), iki farklı anahtarın aynı konuma düşmesidir. 12'yi eklerken 5. konum doludur. Hocanın gösterdiği basit çözüm, elemanı sonraki boş yere koymaktır. 13'te de 6. konum doludur ve sonraki yer dizinin dışına çıkar. Bu yüzden dizi **dairesel (circular)** düşünülür ve eleman 0. konuma konur.

Çakışma varsa arama birkaç adım sürer. 12 aranırken:

1. $12 \bmod 7 = 5$ hesaplanır ve 5. konuma bakılır.
2. Orada 5 vardır, $5 \neq 12$. Sonraki konuma geçilir.
3. 6. konumda 12 bulunur.

Hoca, çakışmaları aşmak için başka teknikler de olduğunu, ama bunların veri yapıları ve algoritma analizi derslerinin konusu olduğunu söyledi. Bu derste yalnız koleksiyonun nasıl kullanılacağına bakılır.

> **Ek bilgi (derste anlatılmadı):** Derste gösterilen "sonraki boş yere koy" yöntemine doğrusal yoklama (linear probing) denir. Java'nın `HashMap`'i ise farklı çalışır: javap ile bakıldığında dizisinin `HashMap$Node` türünde olduğu ve düğümde `next` alanı bulunduğu görüldü, yani çakışan elemanları büyük olasılıkla düğüm zinciriyle tutar. Aynı `hashCode` değerine sahip iki farklı anahtar javac ile denendi ve ikisi de ayrı ayrı saklandı.

3. HashMap Kullanımı

Java'da nesneler bir anahtar (key, benzersiz tanımlayıcı) ve bir değer (value) olarak saklanabilir. Hoca bunu veritabanına benzetti: Veritabanında öğrenci numarası ya da kimlik numarası gibi benzersiz bir bilgi girersiniz, diğer bilgiler gelir. Burada da anahtara ulaşınca nesneye ulaşmış olursunuz. Arka planda ise veritabanı değil, hash veri yapısı vardır.

Sınıf ilişkisi şöyledir:

```
Map (arayüz)
 └─ AbstractMap (Map'i implement eder)
     └─ HashMap (AbstractMap'ten kalıtım alır)
```

`HashMap` thread-safe değildir, yani yarış durumu (race condition) oluşabilir ve karşılıklı dışlama (mutual exclusion) sorunu çıkabilir. Hoca, `Hashtable`'ın thread-safe olduğunu, bunların ayrıntısının ileride çoklu iş parçacığı (multi-threading) konusunda görüleceğini söyledi.

3.1 HashMap Metotları

Tanımda iki tip parametresi vardır: `HashMap<K, V>`. `K` anahtarı, `V` değeri temsil eder. Örneğin anahtarlar birer indeks gibi, değerler de kitap ya da başka nesneler olabilir.

- `get(key)`: Anahtara karşılık gelen nesneyi getirir. Dönüş tipi değer tipine göre değişir (kitap, tamsayı, string, boolean vb.).
- `put(key, value)`: Ekleme işlemidir, anahtar ile değeri ilişkilendirir. Anahtar haritada varsa eski değer döndürülür, yerine yenisi yazılır.

- `values()`: Saklanan bütün değerlerin bir listesini verir. For-each döngüsüyle kolayca gezilir.
- `size()`: Eleman sayısını verir.

`put` davranışı javac ile denendi: Boş haritada `put("k", "v1")` sonucu `null`, ardından `put("k", "v2")` sonucu `"v1"` oldu ve `get("k")` `"v2"` döndürdü. Olmayan bir anahtar için `get` `null` döndürdü.

Hoca ayrıca kullanım için üst düzey arayüzün bütün ayrıntılarını bilmenin gerekmediğini aktardı.

3.2 Course Örneği

Slayttaki örnekte bir `Course` sınıfı öğrencilerini `HashMap` içinde tutar. Sınıfın özel üyeleri ders kodu, ders adı ve kontenjandır. Nesne oluşturulunca constructor çalışır ve `HashMap` başlatılır.

Hocanın hayati dediği noktalar şunlardır: `HashMap`'in tanımlanması, başlatılması, boyutunun alınması. `addStudent` önce iki şartı kontrol eder: Kontenjan dolu mu? Bu öğrenci daha önce eklenmiş mi? Şart ihlal edilirse `false` döner. Değilse öğrenci eklenir. **Öğrenci numarası anahtar verilir, çünkü benzersizdir.** Değer olarak öğrenci nesnesi konur.

Hoca, arka planda illa mod kullanılmayacağını, modun yalnız basit bir örnek olduğunu, ama arka plandakini tam bilmediğimizi ve muhtemelen modun farklı bir biçiminin kullanıldığını söyledi.

`findStudent` metodu `get`'e dayanır. Sonuç bir öğrenci nesnesi ya da `null` referansıdır. `increaseCapacity` ise `Course` sınıfının metodudur, `HashMap` ile ilgisi yoktur. `showClassList` metodu `values()` ile bütün öğrencileri alır ve for-each ile numara ile adı yazar. For-each içindeki `student` değişkeni döngü gövdesi için geçici bir değişkendir.

Aşağıdaki kod anlatımdan ve kısa nottan kurulmuştur. Ders kodu, numara, ad ve kontenjan değerleri ile "Problem" iletileri temsilîdir. Kod javac ile derlenip çalıştırıldı.

```
import java.util.HashMap;

class Student {
    private String number, name;
    public Student(String number, String name) {
        this.number = number;
        this.name = name;
    }
    public String getNumber() { return number; }
    public String getName() { return name; }
}

class Course {
    private String code, name;
    private int capacity;
    private HashMap<String, Student> students;

    public Course(String code, String name, int capacity) {
        this.code = code;
        this.name = name;
        this.capacity = capacity;
        students = new HashMap<String, Student>();
    }
    public String getCode() { return code; }
    public String getName() { return name; }
    public int getCapacity() { return capacity; }
    public int getStudentCount() { return students.size(); }

    public boolean addStudent(Student s) {
        if (getStudentCount() == capacity
```

```
                || findStudent(s.getNumber()) != null)
            return false;
        students.put(s.getNumber(), s);  // anahtar: öğrenci numarası
        return true;
    }
    public Student findStudent(String number) {
        return students.get(number);     // yoksa null
    }
    public void increaseCapacity(int amount) { capacity += amount; }
    public void showClassList() {
        for (Student s : students.values())
            System.out.println(s.getNumber() + " " + s.getName());
    }
}

public class TestCourse {
    public static void main(String[] args) {
        Course course = new Course("KOD", "Ders", 2);
        Student s1 = new Student("N1", "Ogrenci A");
        Student s2 = new Student("N2", "Ogrenci B");
        Student s3 = new Student("N3", "Ogrenci C");

        if (!course.addStudent(s1)) System.out.println("Problem 1");
        if (course.addStudent(s1)) System.out.println("Problem 2");
        if (!course.addStudent(s2)) System.out.println("Problem 3");
        if (course.addStudent(s3)) System.out.println("Problem 4");
        if (course.findStudent("N1") != s1)
            System.out.println("Problem 5");
        course.showClassList();
        System.out.println("End of test");
    }
}
```

Çıktı:

```
N1 Ogrenci A
N2 Ogrenci B
End of test
```

Testte `s1`'in ikinci kez eklenmesi ve dolu derse `s3`'ün eklenmesi `false` döndürdü. `findStudent` bir referans döndürdüğü için sonuç `==` ile eklenen nesneyle karşılaştırılabilir ve iki referans aynı çıktı.

> **Hocanın vurgusu:** `HashMap` önemlidir. Veritabanı tarzı bir uygulamada her kaydın benzersiz bir tanımlayıcısı olur ve kayda ait diğer bilgiler saklanır. Yapısal programlama dersinde projede yapılan bu iş, `HashMap` ile çok rahat yapılır.

4. Java Collection Framework

İstisnalara geçmeden önce hoca, ders notlarında olmayan ama önemli gördüğü koleksiyon ayrıntılarını anlattı.

4.1 Framework Ne Sağlar?

Collection Framework, veri yapıları dersinde görülen yapıları ve algoritmaları hazır bir paket olarak kullanmayı sağlar. C'de böyle bir şey yoktur, kendiniz yazarsınız. Hoca bunun değersiz olmadığını, mantığı öğrenmek için veri yapıları dersinin bu açıdan çok kıymetli olduğunu söyledi. Ama büyük bir projede öğrendiğiniz şeyleri tekrar tekrar yazmamanız için C++ ve Java'da bu tanımlar hazırdır.

Bu yapılar gelişigüzel yazılmamıştır. "Hadi bir stack sınıfı yazalım" denip oluşturulmamışlardır. Nesneye yönelik düşünceye bağlı, belli bir hiyerarşi içinde kurulmuşlardır. İçinde arayüzler,

bunların implementasyonları, ilgili sınıflar ve sıralama ile arama gibi algoritmalar vardır.

4.2 Hiyerarşi

Hocanın gösterdiği şemada sarı kutular arayüz, kırmızı kutular sınıftı. Düz metin olarak şöyledir:

```
Iterable (arayüz)
 └─ Collection (arayüz)
     ├─ List  (arayüz)
     ├─ Queue (arayüz)
     └─ Set   (arayüz)
```

4.3 List ile Set Arasındaki Fark

Hoca soruyu matematikteki kümelerden hareketle sordu: Liste ile küme arasında ne fark vardır? Cevap şudur: Aynı eleman kümeye iki kez eklenemez, listeye eklenebilir. Bir kümenin elemanları liste ile de saklanabilir, ama `Set` ikinci kez eklemeyi engelleyecek biçimde tasarlanmıştır.

> **Ders sonrası not:** Hoca "sıralanmamıştan kastımız aynı elemanı iki defa ekleyemiyoruz" dedi; bunlar farklı özelliklerdir. `Set`'in ayırt edici özelliği tekrarlı eleman kabul etmemesidir. `HashSet` ile javac ile denendi: Aynı elemanın ilk `add` çağrısı `true`, ikincisi `false` döndürdü ve kümede tek eleman kaldı.

4.4 Collection Arayüzünün Metotları

- `add`: Bir eleman ekler.
- `addAll`: Belirtilen koleksiyonun bütün elemanlarını başka bir koleksiyona ekler. Hoca, ilk derslerde dizide kitap eklemeyle ilgili soru soran bir öğrenciye gönderme yaparak iki listeyi birleştirmenin çok kullanışlı olduğunu söyledi.
- `remove`: Bir elemanı siler.
- `removeAll`: Belirtilen koleksiyondaki elemanları siler.
- `retainAll`: `removeAll`'un tersidir. Belirtilen koleksiyondakileri tutar, diğerlerini siler.
- `size`, `clear`: Boyut verir, temizler.
- `contains`: Bir elemanı arar.
- `containsAll`: Bir listenin bütün elemanlarının başka bir listede olup olmadığını kontrol eder.
- `iterator`: Gezinmek için iterator verir.
- `toArray`: Herhangi bir tipteki listeyi `Object` dizisine çevirir. Öğrenci listesi ya da tamsayı listesi fark etmez.
- `isEmpty`: Boş olup olmadığını söyler.
- `equals`: İki koleksiyonun eşleşip eşleşmediğini kontrol eder.
- `hashCode`: Hash ile ilgili hash kodu numarasını döndürür.

`removeAll` ile `retainAll` farkını hoca ekranda bir listeyle gösterdi: Liste içinden 1, 2 ve 3 verilip `removeAll` denirse bunlar silinir. `retainAll` denirse bunlar kalır, diğerleri silinir. Slayttaki renk örneğinde de renk listesine `green` verilince `retainAll` sonrası yalnız `green` kalır.

Kısa nottaki `[1, 2, 3, 4]` listesi üzerinde `[2, 3]` verilerek javac ile denendi: `retainAll` sonucu `[2, 3]`, `removeAll` sonucu `[1, 4]` oldu.

4.5 Iterator Arayüzü

`Iterator`, listedeki bütün elemanları gezmeyi sağlayan arayüzdür. `hasNext` hâlâ eleman olup olmadığını söyler, `next` geçerli elemanı okuyup sonrakine geçer.

> **Ders sonrası not:** Hoca iterator'ın "listede ya da map'te" gezdiğini söyledi; `Map` doğrudan iterator vermez. `HashMap` üzerinde `iterator()` çağrısı javac ile denendi ve `cannot find symbol` hatası verdi. `HashMap`'in `Collection` türünden olmadığı da java ile doğrulandı. Map'te gezmek için derste gösterilen `values()` kullanılır.

5. ArrayList ve Iterator

Hoca `ArrayList`'in eski, generic olmayan tanımının artık kullanılmayan bir biçim olduğunu söyledi. Yeni biçimde generic tanım kullanılır: Küçüktür ve büyüktür işaretleri arasına `String` ya da herhangi bir nesne tipi yazılır. `add` metodu `Collection` arayüzünden gelir ve `ArrayList` içinde implement edilir, çünkü arayüzü implement eden sınıf bunu tamamlamak zorundadır.

Soru: For-each döngüsü her çalıştığında arka planda iterator mı kullanıyor?

Cevap: Hoca "aslında tersi, iterator kullandığınızda arka planda for-each çalışıyor" dedi.

> **Ders sonrası not:** Hoca "iterator arka planda for-each kullanır" dedi; doğrusu öğrencinin söylediğidir. Liste üzerindeki for-each döngüsü javac ile derlenip javap ile incelendi: Üretilen kodda `List.iterator`, `Iterator.hasNext` ve `Iterator.next` çağrıları görüldü. Dizi üzerindeki for-each ise iterator kullanmıyor, aynı incelemede `arraylength` ile indeksli gezinme görüldü.

`ArrayList`'in `Collection`'a ek metotları şunlardır:

- `lastIndexOf(nesne)`: Elemanın son geçtiği konumu verir.
- `indexOf(nesne)`: Elemanın ilk geçtiği konumu verir.
- `trimToSize()`: `ArrayList`'in kapasitesini küçültür.

Hocanın örneği, aynı değerin birkaç kez geçtiği bir tamsayı listesiydi: `1, 2, 1, 3, 1, 1`. Konumlar sıfırdan başlar (ara adım eklendi). 1 değeri 0, 2, 4 ve 5. konumlardadır. Bu yüzden `indexOf(1)` sonucu 0, `lastIndexOf(1)` sonucu 5 olur. İki sonuç da javac ile denendi.

5.1 For-each ile Iterator

Slayttaki ilk örnekte bir `ArrayList<String>` tanımlanır, elemanlar eklenir ve bunlar hem for-each hem iterator ile gezilir. Hoca, iterator'ın tipten bağımsız bir yaklaşım olduğunu, for-each'te ise temel nesne tipinin belirtilmesi gerektiğini söyledi.

> **Ders sonrası not:** Hoca iterator'da tip belirtmeye gerek olmadığını söyledi; bu eksiktir. Slayttaki gibi `Iterator<String>` yazıldığında tip belirtilmiş olur. Tip yazılmazsa `next` `Object` döndürür; `String s = it.next();` satırı javac ile denendi ve `incompatible types: Object cannot be converted to String` hatası verdi.

5.2 Öğrenci Listesi Örneği

İkinci örnekte öğrenci tipinde bir liste oluşturulur, öğrenciler eklenir. Ardından ikinci bir öğrenci listesi `addAll` ile birinci listeye eklenerek iki liste birleştirilir. Bir `listeGoster` metodu iterator ile listeyi gezer. `hasNext`, hâlâ nesne varsa dönmeye devam et demektir. Her `next` bize bir öğrenci nesnesi döndürür.

Slayttaki kodda öğrencinin üyelerine tek tek erişilip ekrana basılıyordu. Hoca bunu yazmamak gerektiğini söyledi: Bu iş `toString` metoduna yazılırsa o satır kurtulur ve doğrudan `System.out.println(iterator.next())` denir. Aşağıdaki kod hocanın önerdiği biçimdedir. Adlar ve numaralar temsilîdir. Kod javac ile derlenip çalıştırıldı.

```
import java.util.ArrayList;
import java.util.Iterator;

class Ogrenci {
    private String no, ad;
    public Ogrenci(String no, String ad) {
        this.no = no;
        this.ad = ad;
    }
    public String toString() { return no + " " + ad; }
}

public class TestListe {
    static void listeGoster(ArrayList<Ogrenci> liste) {
```

```
        Iterator<Ogrenci> it = liste.iterator();
        while (it.hasNext())
            System.out.println(it.next());
    }
    public static void main(String[] args) {
        ArrayList<Ogrenci> liste1 = new ArrayList<Ogrenci>();
        liste1.add(new Ogrenci("N1", "Ogrenci A"));
        ArrayList<Ogrenci> liste2 = new ArrayList<Ogrenci>();
        liste2.add(new Ogrenci("N2", "Ogrenci B"));
        liste1.addAll(liste2);      // iki liste birleştirilir
        listeGoster(liste1);
    }
}
```

Çıktı:

```
N1 Ogrenci A
N2 Ogrenci B
```

6. LinkedList ve ArrayList ile Karşılaştırması

Java'da `LinkedList` varsayılan olarak **çift bağlı listedir**. Bu yüzden iki yönde de gezilebilir. `LinkedList` sınıfından yararlanarak stack ve kuyruk (queue) da kurulabilir. Stack'in `push` ve `pop`, kuyruğun `enqueue` ve `dequeue` işlemleri vardır. Listenin başına ya da sonuna ekleyip buradan silerek bu yapılar elde edilir.

Metotlar: `addFirst`, `addLast`, `removeFirst`, `indexOf`, `lastIndexOf`, `remove`, `contains`, `getFirst`, `getLast`. `LinkedList` de generic tanımla herhangi bir nesne tipi için kullanılır.

Hoca aynı anlatımda stack ve kuyruğun nasıl kurulacağını yalnız sözle söyledi. Aşağıdaki kod bu anlatımdan kuruldu, elemanlar temsilîdir. Kod javac ile derlenip çalıştırıldı.

```
import java.util.LinkedList;

public class TestLinked {
    public static void main(String[] args) {
        LinkedList<String> stack = new LinkedList<String>();
        stack.addFirst("a");            // push
        stack.addFirst("b");            // push
        System.out.println(stack.removeFirst());  // pop

        LinkedList<String> kuyruk = new LinkedList<String>();
        kuyruk.addLast("a");            // enqueue
        kuyruk.addLast("b");            // enqueue
        System.out.println(kuyruk.removeFirst()); // dequeue
    }
}
```

Çıktı:

```
b
a
```

6.1 Jenerik ve Arayüz Hatırlatması

Hoca slaytlardan geçerken daha önce işlenen iki örneği hatırlattı. Birincisi jeneriklerdir: Jenerik sınıf ve metot tanımlamayı görmüştük. Birden fazla jenerik tip yazılacaksa `T` ve `U` gibi farklı semboller kullanılır.

İkincisi arayüz kısmındaki veritabanı örneğidir. Kitap sınıfı vardı, bir de ekleme ve silme yapacak bir veritabanı arayüzü. MySQL de olsa MSSQL de olsa bu şartı sağlamak zorundadır. Kütüphanemiz hangi veritabanını kullanacaksa onu kullanır, diğer her şey aynı kalır. Veritabanını değiştirmek istediğimizde yalnız o noktayı değiştirmemiz gerekir.

6.2 ArrayList ile LinkedList Farkı

Özellik	`ArrayList`	`LinkedList`
İç yapı	Dinamik dizi, C'deki `malloc` gibi	Elemanlar arasında referans bağları
Ekleme ve silme	Yavaş, kaydırma gerekir	Daha hızlı, kaydırma gerekmez
Davranış	Yalnız liste, `List`'i implement eder	Hem liste hem kuyruk
Uygun olduğu iş	Saklama ve erişim	Veri üzerinde değişiklik

> **Ders sonrası not:** Hoca `LinkedList`'in dinamik bellek kullanmadığını söyledi; asıl fark bu değildir, fark bitişik dizi ile bağlı düğümlerdir. javap ile bakıldığında `ArrayList`'te `Object[] elementData`, `LinkedList`'te `first` ve `last` düğümleri ile `next` ve `prev` alanlı `LinkedList$Node` sınıfı görüldü. Düğümler de büyük olasılıkla çalışma anında tek tek oluşturulur, yani bu da dinamik bellek kullanımıdır.

Hocanın hiyerarşi hakkındaki sözü doğrudur: `LinkedList`'in `List` ve `Deque` arayüzlerini implement ettiği, `ArrayList`'in arayüzleri arasında kuyruk türü olmadığı java ile doğrulandı. Hoca bu farkların veri yapıları dersiyle ilgili olduğunu söyledi.

7. Değeri Liste Olan HashMap: Basit Bir Veritabanı

Hoca başka bir kaynaktaki `HashMap` örneklerini de gösterdi. Bir örnekte anahtar tamsayı, değer string'dir. Sonra önemli bulduğu ve derste anlatılmadığını belirttiği bir noktaya geldi: **Anahtar benzersiz olmalıdır, ama değer bir liste olabilir.**

Örneğin her dersin kodu anahtar, dersi alan öğrencilerin listesi değer olabilir. Hoca bunu ekranda çizdi. Düz metin olarak yapı şöyledir:

```
HashMap
 anahtar (ders kodu)   değer (ArrayList<Student>)
 ders kodu 1   ──────► [ student 0, student 1, ... ]
 ders kodu 2   ──────► [ student 0, student 1, ... ]
```

Listenin her elemanı bir öğrencidir ve adını, soyadını, yaşını, doğum gününü içerir. Bu listeyi anahtar üzerinden `HashMap`'e bağlayarak basit bir veritabanı tasarlanabilir. Hoca, daha önceki yıllarda bununla ilgili bir laboratuvar sorusu sorduğunu da söyledi.

Aşağıdaki kod bu anlatımdan kuruldu, değerler temsilîdir. Kod javac ile derlenip çalıştırıldı.

```java
import java.util.ArrayList;
import java.util.HashMap;

class Student {
    private String name;
    public Student(String name) { this.name = name; }
    public String toString() { return name; }
}

public class TestVeritabani {
    public static void main(String[] args) {
        HashMap<String, ArrayList<Student>> dersler =
                new HashMap<String, ArrayList<Student>>();

        ArrayList<Student> liste = new ArrayList<Student>();
        liste.add(new Student("Ogrenci A"));
        liste.add(new Student("Ogrenci B"));
        dersler.put("KOD", liste);

        System.out.println(dersler.get("KOD"));
```

```
    }
}
```

Çıktı:

```
[Ogrenci A, Ogrenci B]
```

8. İstisna Yönetimine Giriş

Aradan sonra hoca istisna (exception) konusuna geçti. Ders notlarında az anlatıldığını, ama bilinmesi gereken ayrıntılar olduğunu söyledi. Slayt, "hata yapma şansı olan onu yapacaktır" diye özetlenebilecek bir sözle başlıyordu: Bir programda bir hata varsa o hata ortaya çıkacaktır. Program yazarken her ayrıntıyı dikkate almak zordur. Sonuçta hatalar oluşur ve program çöker (crash). Bunun önüne geçmek için istisna yönetimi mekanizmaları kullanılır.

8.1 Hatadan Kaçınma ile İstisna Yönetiminin Farkı

İstisna yönetimi hatadan kaçınmak değil, hata olduğunda onunla baş etmeye çalışmaktır. Hoca farkı bir soruyla gösterdi.

Soru: 0 ile 100 arasında bir sayı girilecek. Hatadan kaçınmak için koda ne eklemek gerekir?

Cevap: Bir öğrenci, `if` koyup girilen sayı 0'dan küçük ya da 100'den büyükse kabul etmemeyi önerdi. Hoca bunu onayladı. Tekrar değer girilmesi istenirse `do-while` ya da `while` döngüsü kullanılır.

Bunlar önceden düşünülüp yazılan önlemlerdir ve düzgün bir algoritma geliştirmeyle ilgilidir. İstisna yönetimi ise başka bir şeydir: Hata olmuş, "iş işten geçmiş" durumdayız, artık sonrasına bakıyoruz. Amaç, programın kapanmaması için bir şeyler yapmaktır.

Hoca, C'de de program çökerken çalışacak ufak bir metot tanımlanabildiğini, ama bunun çok kullanılmadığını söyledi.

8.2 Sınıf Hiyerarşisi

Java'da `Throwable` ile başlayan bir sınıf hiyerarşisi vardır. Aşağıdaki üst sınıf ilişkileri java ile doğrulandı.

```
Throwable
 ├─ Error
 └─ Exception
     ├─ IOException
     └─ RuntimeException
         ├─ ArithmeticException
         ├─ NullPointerException
         ├─ IllegalArgumentException
         │   └─ NumberFormatException
         └─ IndexOutOfBoundsException
             └─ ArrayIndexOutOfBoundsException
```

`Error` altındaki sorunlara hoca şu örnekleri verdi: Sistem kaynaklarının aşırı tüketilmesi, Java Sanal Makinesi'yle ilgili hatalar, desteklenmeyen sınıf sürümleri.

Diğer istisna örnekleri şunlardır:

- **`ArrayIndexOutOfBoundsException`:** 10 elemanlı bir dizinin 11. elemanına yazmaya çalışmak. Hoca, C'de belki bu hatayı almadığımızı, Java'da alacağımızı söyledi.
- **`NullPointerException`:** Bir sınıfta bir kitap referansı tuttunuz, ama ona nesne atamadınız ve onu ekrana basmaya çalışıyorsunuz. Hoca bunun öğrencilerin çok yaptığı bir hata olduğunu söyledi.
- **`IOException`:** Bir dosyaya erişmeye çalışıyorsunuz, ama dosyanın adını yanlış yazdınız.
- **`ArithmeticException`:** Kullanıcıdan iki sayı aldınız, ikinci sayı olarak 0 girildi ve sıfıra bölme oluştu.

Ayrıca kendi istisnalarımızı da tanımlayabiliriz. Hocanın iki örneği şunlardı: “Adı Ahmet olan bu dersi seçemez” gibi bir kural tanımlanır. Böyle biri bir şekilde dersi eklemeye çalışırsa sistem yöneticisine uyarı gönderilir, yani bir koruma mekanizması kurulur. Diğer örnek de “yaşı 18’den küçükler uygun değildir” kuralıdır.

Konuyu anlamak için bu sınıf hiyerarşisini ve beş anahtar kelimeyi iyi bilmek gerekir: `try`, `catch`, `finally`, `throw`, `throws`. Hoca, bunlar bilinince sıkıntı yaşanmayacağını söyledi.

8.3 İlk Deneme: Yakalanmayan İstisna

Hoca bir `5 / 0` ifadesini `System.out.println` içinde çalıştırdı. Program derleme aşamasında sorun çıkarmadı, “Build Successful” dedi. Hata çalışma anında ortaya çıktı: `ArithmeticException` sınıfında sıfıra bölme hatası. Sonra dizinin dışına erişmeyi denedi: 10 elemanlı `a` dizisinde `a[10]`’a erişim `ArrayIndexOutOfBoundsException` verdi. İkisini denerken önce birincisinin programı durdurduğunu, ikinciye geçilemediğini de gösterdi.

İki durum javac ile denendi. Çıktılar şöyledir:

```
Exception in thread "main" java.lang.ArithmeticException: / by zero
Exception in thread "main" java.lang.ArrayIndexOutOfBoundsException:
    Index 10 out of bounds for length 10
```

> **Ders sonrası not:** Hoca, kurulu bir programda istisnanın işletim sistemine fırlatıldığını ve işletim sisteminin bunu size bildirdiğini söyledi; denemedeki durum farklıdır. Yukarıdaki iletiyi Java’nın kendisi yazdı ve süreç 1 çıkış koduyla sonlandı. İşletim sistemi tarafı ise büyük olasılıkla yalnız bu çıkış kodunu görür.

9. try, catch ve finally

Hataya meyilli kısım `try` bloğuna yazılır. Hata olursa sistem otomatik olarak `catch` bloğuna düşer ve `try` içinde kalan satırlar işlenmez. Örneğin 10 satırın 3.’sünde istisna olursa diğer satırlar çalışmaz, ama program kapanmaz; `catch`’ten sonra işlem devam eder. Hoca, hangi istisnanın geldiğini görmek için `catch` içinde `Exception` sınıfından bir `e` referansı alındığını gösterdi.

Aşağıdaki kod hocanın ekranda yazdığı örnektir. Kod javac ile derlenip çalıştırıldı.

```
public class TestTry {
    public static void main(String[] args) {
        try {
            System.out.println("Önce");
            int x = 5 / 0;
            System.out.println("Sonra");
        } catch (Exception e) {
            System.out.println(e.toString());
        }
        System.out.println("Program bitti");
    }
}
```

Çıktı:

```
Önce
java.lang.ArithmeticException: / by zero
Program bitti
```

`Sonra` hiç görünmedi, ama program da kapanmadı.

> **Hocanın vurgusu:** `try` ile `catch` birbirinden ayrılmaz. `try` yazıp `catch` yazmamak olmaz.

> **Ders sonrası not:** Hoca `try`’ın `catch`’siz olmayacağını söyledi; bu eksiktir. Yalnız `try` yazmak javac ile denendi ve `'try' without 'catch', 'finally' or resource declarations` hatası verdi. Ama `try-finally` yazmak derlendi ve çalıştı. Yani `try`’dan sonra en az `catch` ya da `finally` gerekir.

Hoca bu noktada slayttaki bir cümleye de değindi: “A program may opt to not handle a checked exception.” Bir metoda `throws` yazıldığında, metodun örneğin `IOException` fırlatabileceği bildirilir. Bu metodu çağıran taraf çağrıyı `try` içinde kullanmak durumunda kalır. Ayrıntılı örnekler 12. bölümdedir.

> **Ders sonrası not:** Hoca bu cümleyi “programcı checked bir istisnayı handle etmek isteyebilir” diye çevirdi; slayttaki cümle tersini, yani yakalamamayı seçebileceğini söyler. O zaman istisna `throws` ile bildirilmelidir. Çağıran taraf için de `try` tek seçenek değildir, kendisi de `throws` yazabilir; bu javac ile denendi ve derlendi.

9.1 Birden Fazla catch

Birden fazla hata olasılığı olabilir: Birinden kurtulursunuz, öbürü oluşur. Hoca örneği `5 / 1` yaparak bölme hatasını ortadan kaldırdı, sonra bir dizi oluşturup 5. elemana 9 yazdı. İstisnalar hiyerarşik olarak art arda yazılabilir. Program bölmeden geçer, dizide hata alır. `catch (Exception e)` genel üst sınıf olduğu için bu durumda da çalışır, sadece gelen mesaj farklıdır.

Hocanın söylediği kullanım şudur: Kısa bir algoritmada iki olası hata varsa, her işlem için ayrı `try` yazmak yerine bir `try` ve iki `catch` yazılabilir.

Aşağıdaki kod hocanın örneğinden kurulmuştur. Dizi boyutu altyazıda okunmadığı için, çıktının aynı istisnayı vermesi için 5 alındı. Kod javac ile derlenip çalıştırıldı.

```
public class TestCokluCatch {
    public static void main(String[] args) {
        try {
            int x = 5 / 1;
            System.out.println("Bir");
            int[] a = new int[5];
            a[5] = 9;
            System.out.println("Iki");
        } catch (ArithmeticException e) {
            System.out.println("Aritmetik: " + e.getMessage());
        } catch (Exception e) {
            System.out.println(e.toString());
        } finally {
            System.out.println("Close file access");
        }
        System.out.println("Program bitti");
    }
}
```

Çıktı:

```
Bir
java.lang.ArrayIndexOutOfBoundsException: Index 5 out of bounds for length 5
Close file access
Program bitti
```

Hoca bu sırada `catch` içine `IOException` yazmayı denedi, hata aldı ve bunu “RuntimeException kapsıyor, izin vermiyor” diye yorumladı. Sonra “biz biraz önce bu IOException’ı niye yakalayamadık” diye soruyu açık bıraktı.

> **Ders sonrası not:** Hoca hatayı `RuntimeException`’ın kapsamasına bağladı; asıl sebep başkadır. `IOException` checked bir istisnadır ve `try` gövdesinde onu fırlatabilecek bir ifade yoktur. javac ile denendi: Import edilmezse `cannot find symbol`, edilirse `exception IOException is never thrown in body of corresponding try statement` hatası alındı. Hocanın sonraki bölümde ekranda gördüğü hata da budur.

9.2 printStackTrace

`e.printStackTrace()` istisnaların hangi sırayla oluştuğunu yazar. İç içe birçok istisna olabilir ve bunlar böyle loglanabilir. Hoca bunun çok önemli olmadığını, ama bazı yerlerde kullanıldığını söyledi.

9.3 finally

İstisna oluştuğunda programı kapatmanız gerekiyorsa kaynakları bırakmak istersiniz: Veritabanı bağlantısını, dosya erişimini kapatmak, değişkenler için çöp toplayıcıyı çağırmak gibi. Bunlar `catch` içinde yapılabilir. Ama hata olmazsa da aynı işler yapılacağından, kodu iki yere yazmak gerekir. Daha güzeli `finally` bloğudur. **Hata olsun ya da olmasın** çalışan işler buraya yazılır. Hoca bunu "Close file access" iletisiyle gösterdi.

9.4 Slayt Örneği: parseMyCatFile

Slaytta `parseMyCatFile` adlı bir metot vardı. Metot bir dosya adı alır, dosyayı açar, içindeki resimleri bir diziye alır, `drawFigure` ile çizer ve dosyayı kapatır. Hoca burada resim açma uygulamasının olmadığını, bunun yalnız bir kurgu olduğunu belirtti. Binlerce resim açarken dosya adlarından biri yanlış yazılmış olabilir. O zaman dosya açılamaz, okunamaz ve bu satırlardan birinde hata olur. Bu yüzden blok `try` içindedir.

Hata IO sırasında oluştuğu için program kapanmasın diye `catch`'e gelinir ve `System.out.println` ile hangi dosyanın açılamadığı yazılır. `finally` kısmında ise kaynaklar bırakılır: Bir sürü resim bellekte çok yer kapladığı için belleği şişirmemek adına `releaseResource` benzeri bir metot çağrılır ya da dosya kapatılır.

> **Ek bilgi (derste anlatılmadı):** `finally`'nin çalışmadığı bir durum vardır: `try` içinde `System.exit` çağrılırsa program hemen sonlanır. javac ile denendi: `try` içindeki `System.exit(0)` çağrısından sonra `finally`'deki satır yazılmadı.

10. Kendi İstisna Sınıfımızı Tanımlamak

Kendi istisna sınıfımızı yazmak için `IOException` ya da başka bir istisna sınıfından kalıtım almak zorundayız. Sınıfın constructor'ında `super` yazılır ki üst sınıfın olanakları kullanılabilsin. Slaytta `MyFileFormatException` adlı bir sınıf vardı: Bir hata mesajı alır ve bunu üst sınıfa gönderir.

Bir metot bu istisnayı `throws` ile bildirir. Metodun içinde beklenmeyen bir durum `if` ile kontrol edilir ve yeni bir anahtar kelimeyle, `throw` ile istisna oluşturulur. Hoca burada öğrencinin hatadan kaçınma için söylediği `if`'e gönderme yaptı.

Slayttaki kurgu şöyle yazılabilir. Kod javac ile derlendi.

```
import java.io.IOException;

class MyFileFormatException extends IOException {
    public MyFileFormatException(String errorMessage) {
        super(errorMessage);
    }
}

public class Program {
    void processFile(boolean beklenmeyenDurum)
            throws MyFileFormatException {
        // some statements
        if (beklenmeyenDurum)
            throw new MyFileFormatException("hata mesaji");
    }
}
```

> **Hocanın vurgusu:** `throws` metodun tanımında kullanılır. `throw` hatayı üretmek için `if` ile birlikte ve `new` ile kullanılır, nesne üretir.

10.1 ImpossibleInfo Örneği

Slayt bütün bunları bir örnekte birleştiriyordu. `ImpossibleInfo` sınıfı `IOException`'dan türer. Hata mesajını alır ve üst sınıfa gönderir. Kişi sınıfında yaşı ayarlayan metot bu türden bir

istisna fırlatabileceğini bildirir. Yaş 0'dan küçük ya da 150'den büyükse `throw new ImpossibleInfo(...)` çalışır.

`new` ile nesne üretmek aslında constructor'ı çağırmaktır. Mesaj önce bu constructor'a, oradan `IOException`'a gider.

`main` içinde `setAge` artık `try` içinde kullanılır, çünkü istisna olma ihtimali vardır. `catch` bu türü yakalar ve `printStackTrace` çağırır. Ardından klavye girişi (`input`) kapatılır. Hata yoksa sorun olmaz. Hata olursa bizim yazdığımız `Impossible age` mesajı girilen yaş değeriyle birlikte görünür. Mesaj normal çıktıdan farklı olarak kırmızı görünür.

Aşağıdaki kod anlatımdan kuruldu. Aralık dışı ve aralık içi bir girdiyle javac ile derlenip çalıştırıldı.

```
import java.io.IOException;
import java.util.Scanner;

class ImpossibleInfo extends IOException {
    public ImpossibleInfo(String message) { super(message); }
}

class Person {
    private int age;
    public void setAge(int age) throws ImpossibleInfo {
        if (age < 0 || age > 150)
            throw new ImpossibleInfo("Impossible age: " + age);
        this.age = age;
    }
    public int getAge() { return age; }
}

public class TestImpossibleInfo {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        Person person = new Person();
        try {
            person.setAge(input.nextInt());
            System.out.println("Age: " + person.getAge());
        } catch (ImpossibleInfo e) {
            e.printStackTrace();
        }
        input.close();
    }
}
```

Aralık dışında bir yaş girilince hata akışına `ImpossibleInfo: Impossible age: <girilen yaş>` satırı ve ardından çağrı yığını yazıldı, program normal biçimde bitti.

11. Hata Sonrası Akış, Çoklu catch ve İç İçe try

Hoca sonraki slayt setine geçti. Slaytta art arda gelen ifadeler vardı: Statement 5'te bir hata olursa statement 6, 7, 8 çalışır mı? Hayır. Biz devam etmesini istiyoruz.

11.1 Statement 5'te Neler Olabilir?

Hoca dört örnek saydı: Aritmetik hata, `null` bir string'in uzunluğunu almak, `"abc"` metnini tamsayıya çevirmek, dizinin dışına erişmek. Dördü de javac ile denendi. Sayı ve dizi değerleri temsilîdir.

```
public class TestStatement5 {
    public static void main(String[] args) {
        try {
```

```
            int x = 30 / 0;
        } catch (ArithmeticException e) {
            System.out.println("ArithmeticException");
        }
        try {
            String s = null;
            s.length();
        } catch (NullPointerException e) {
            System.out.println("NullPointerException");
        }
        try {
            int y = Integer.parseInt("abc");
        } catch (NumberFormatException e) {
            System.out.println("NumberFormatException");
        }
        try {
            int[] a = new int[5];
            a[10] = 1;
        } catch (ArrayIndexOutOfBoundsException e) {
            System.out.println("ArrayIndexOutOfBoundsException");
        }
    }
}
```

Çıktı:

```
ArithmeticException
NullPointerException
NumberFormatException
ArrayIndexOutOfBoundsException
```

11.2 Slaydın Çoklu catch Örneği

Örnekte bir dizi tanımlanır, sonra `a[5] = 30 / 0` çalışır. Önce `30 / 0` hesaplanır, sonra sonuç dizinin 5. elemanına atanacaktır. Bölme sırasında istisna fırlatılır ve bizi aritmetik `catch` bloğuna götürür. Atama hiç yapılmaz. Diğer `catch` bloklarına girilmez, çünkü onların istisnaları oluşmadı. Sonra “rest of the code” görülür.

`30 / 1` yazılsaydı bölme başarılı olur, atamada dizi sınırı aşılır ve ikinci `catch`’e düşülürdü. Yine “rest of the code” görülürdü.

Kısa nottaki gibi dizi boyutu 5 alındı. “task 1 completed” dışındaki iletiler temsilîdir. İki durum da javac ile derlenip çalıştırıldı.

```
public class TestSlaytCatch {
    public static void main(String[] args) {
        try {
            int[] a = new int[5];
            a[5] = 30 / 0;
        } catch (ArithmeticException e) {
            System.out.println("task 1 completed");
        } catch (ArrayIndexOutOfBoundsException e) {
            System.out.println("task 2 completed");
        } catch (Exception e) {
            System.out.println("common task completed");
        }
        System.out.println("rest of the code");
    }
}
```

Çıktı (`30 / 0` için):

```
task 1 completed
rest of the code
```

`30 / 1` için ilk satır `task 2 completed` oldu.

11.3 catch Sırası: Özelden Genele

Hoca, daha önce belki yanlış yazdığını belirterek kuralı söyledi: `catch` blokları **özelden genele** doğru yazılır. `Exception` önce yazılırsa, o zaten altındaki sınıfları kapsar ve sonraki `catch` gereksizleşir. Hoca bunu ekranda gösterdi.

Aşağıdaki kod javac ile denendi ve `exception ArithmeticException has already been caught` hatası verdi.

```
public class TestSira {
    public static void main(String[] args) {
        try {
            int x = 5 / 0;
        } catch (Exception e) {           // Derleme hatası
            System.out.println("genel");
        } catch (ArithmeticException e) {
            System.out.println("ozel");
        }
    }
}
```

Hocanın vurgusu: Birden fazla `catch` yazarken sıra özelden genele olmalıdır.

11.4 İç İçe try

Bir `try`'ın içinde başka bir `try` yazılabilir. Hocanın örneğinde dışta `Exception` yakalayan bir `try` vardır. İçeride önce bölme hatası yakalanır, sonra bir dizi oluşturulur ve sınır hatası yakalanır. Her iki iç hata kendi `catch`'inde çözüldüğü için dış `catch`'e düşülmez. Program normal akışına devam eder.

Bölme ifadesi ve dizi değerleri ile "normal flow" iletisi temsilîdir. Kod javac ile derlenip çalıştırıldı.

```
public class Exception1 {
    public static void main(String[] args) {
        try {
            try {
                System.out.println("bolme");
                int b = 5 / 0;
            } catch (ArithmeticException e) {
                System.out.println(e);
            }
            try {
                int[] a = new int[5];
                a[5] = 4;
            } catch (ArrayIndexOutOfBoundsException e) {
                System.out.println(e);
            }
            System.out.println("other statement");
        } catch (Exception e) {
            System.out.println("handled");
        }
        System.out.println("normal flow");
    }
}
```

Çıktı:

```
bolme
java.lang.ArithmeticException: / by zero
java.lang.ArrayIndexOutOfBoundsException: Index 5 out of bounds for length 5
other statement
normal flow
```

Hocanın çıktıda gösterdiği gibi `handled` yazılmadı.

11.5 finally'nin Üç Durumu

Slayt `finally` için üç durum sayıyordu. Hoca ikinci durumu şöyle açıkladı: İstisna oldu ama yakalanmadı demek, yanlış bir `catch` yazıldı demektir. Örneğin oluşan istisna sıfıra bölmedir, ama yazılan `catch` `NullPointerException` içindir. Ya doğru istisna ya da genel `Exception` yazmak gerekir.

Durum	`finally` çalışır mı?	"rest of the code" görülür mü?
İstisna yok	Evet	Evet
İstisna var, yakalanmadı	Evet	Hayır, program sonlanır
İstisna var, yakalandı	Evet	Evet

Tablodaki üç durum, aşağıdaki kodun bölme ifadesi ve `catch` türü değiştirilerek javac ile denendi. Kod ikinci durumu gösterir.

```
public class TestFinally {
    public static void main(String[] args) {
        try {
            int x = 30 / 0;
        } catch (NullPointerException e) {
            System.out.println(e);
        } finally {
            System.out.println("finally");
        }
        System.out.println("rest of the code");
    }
}
```

Çıktı:

```
finally
Exception in thread "main" java.lang.ArithmeticException: / by zero
```

12. throw ve throws

12.1 throw: İstisna Üretmek

Hoca `validate` metodunu sıfırdan yazdı. Örnek oy kullanma yaşıyla ilgilidir. Metot yaşı alır. 18'den küçükse `throw new ArithmeticException("not valid")` ile istisna üretir, değilse "welcome" yazar. Hoca, bir şeyin istisna olup olmadığını kontrol ederken mantıken bir `if` gerektiğini söyledi.

Anahtar kelimeleri karıştırmamak için İngilizceden hareketle bir hatırlatma verdi: `throws` geniş zamandır, "fırlatır, fırlatabilir" anlamında bir bildirimdir. `throw` emirdir, "fırlat" anlamındadır, yani hatayı üretir.

```
public class TestThrow1 {
    static void validate(int age) {
        if (age < 18)
            throw new ArithmeticException("not valid");
        else
            System.out.println("welcome");
    }
```

```
    public static void main(String[] args) {
        validate(13);
        System.out.println("rest of the code");
    }
}
```

Soru: Program nasıl çalışır?

Cevap: Öğrenci, yaş 18'den küçük olduğu için istisna üretileceğini, ardından "rest of the code" yazılacağını söyledi. Hoca "istisnayı yakaladık mı?" diye sordu. Yakalanmadı, üretildi. Bu yüzden program "tabiri caizse patlar": `ArithmeticException: not valid` görülür ve program devam etmez, çünkü çağrı `try` içinde yazılmadı.

Kod javac ile derlenip çalıştırıldı ve hocanın dediği görüldü:

```
Exception in thread "main" java.lang.ArithmeticException: not valid
```

Çağrı `try` içine alınır ve `catch`'te `e.toString()` yazdırılırsa çıktı şu olur. Bu değişiklik de javac ile denendi.

```
java.lang.ArithmeticException: not valid
rest of the code
```

Hoca sonra `e.printStackTrace()` ekledi. Bu, program çalışırken oluşan hataların loglanması gibidir. Ekranda hatanın hangi satırlardan geçerek geldiği görüldü, ama program doğru biçimde tamamlandı.

12.2 throws ve Checked ile Unchecked Ayrımı

`throws` bir bildirimdir: Bu metot şu türde istisna fırlatabilir. Bir metoda `throws` yazıyorsanız türünü de belirtirsiniz. Kendi yazdığımız bir sınıf da burada kullanılabilir.

Hoca `try-catch`'i silip `throws ArithmeticException` yazdı. Derleme hatası almadı. Sonra bunu `throws Exception` ve `throw new Exception(...)` yaptı. Bu kez çağrı satırında **unreported exception** hatası geldi.

Neden biri olur da diğeri olmaz? Hoca bunu checked ve unchecked ayrımıyla açıkladı. Bazı istisnalar tanımı gereği `try` ile çevrelenmelidir. `RuntimeException` unchecked'tir. Onu yazınca derleyici uyarmaz.

Aşağıdaki kod javac ile denendi ve `unreported exception Exception; must be caught or declared to be thrown` hatası verdi. `Exception` yerine `RuntimeException` yazılınca kod derlendi.

```
public class TestThrows {
    static void validate(int age) throws Exception {
        if (age < 18)
            throw new Exception("not valid");
    }
    public static void main(String[] args) {
        validate(13);                 // Derleme hatası
        System.out.println("rest of the code");
    }
}
```

> **Ders sonrası not:** Slayt okunurken altyazıda "IOUnchecked" geçiyor ve hoca sonra "IOException zorunlu olmadığı için try bloğuna gerek yok" dedi; `IOException` checked'tir. Aşağıdaki slayt örneğinde `try` gerekmemesinin sebebi `main`'in de `throws IOException` bildirmesidir. `Error` ve `RuntimeException` unchecked'tir; java ile bakıldığında `IOException`'ın üst sınıfının doğrudan `Exception` olduğu görüldü.
>
> **Ders sonrası not:** Hoca, kodu `throws` ile çalıştırınca "demek ki bize bunu sağlıyormuş" dedi; programın devam etmesini sağlayan `throws` değil, `try-catch`'tir. Ekranda büyük olasılıkla hâlâ önceki `try`'lı derleme çalışıyordu, hoca da "biraz önce diğerini çalıştırdı" dedi. Yalnız `throws ArithmeticException` yazıp `try` kullanmamak javac ile denendi: Program istisnayla sonlandı ve "rest of the code" yazılmadı.

12.3 Slayttaki Üç throws Örneği

Birinci durumda metot koşulsuz olarak `device error` mesajlı bir `IOException` fırlatır, çağıran taraf bunu `try-catch` içinde yakalar. `e`'nin mesajı yazdırılsaydı "device error" görülecekti. Burada "exception handled" yazılır ve normal akış devam eder.

```
import java.io.IOException;

class M {
    void method() throws IOException {
        throw new IOException("device error");
    }
}

public class TestThrowsA {
    public static void main(String[] args) {
        try {
            M m = new M();
            m.method();
        } catch (Exception e) {
            System.out.println("exception handled");
        }
        System.out.println("normal flow");
    }
}
```

Çıktı:

```
exception handled
normal flow
```

İkinci durumda metot `throws IOException` bildirir, ama içinde `throw` yoktur. `main` de `throws IOException` bildirir ve `try` yoktur. İstisna oluşmadığı için kod sorunsuz çalışır.

```
import java.io.IOException;

class M {
    void method() throws IOException {
        System.out.println("device operation performed");
    }
}

public class TestThrowsB {
    public static void main(String[] args) throws IOException {
        M m = new M();
        m.method();
        System.out.println("normal flow");
    }
}
```

Çıktı:

```
device operation performed
normal flow
```

Üçüncü durumda aynı `main` korunur, ama metodun içinde birinci durumdaki koşulsuz `throw` vardır. İstisna kesin oluşur ve yakalanmadığı için programın akışı kesilir.

> **Ders sonrası not:** Hoca üçüncü durumda "device error" bile yazılamayacağını, "runtime exception" görüleceğini ve sonunda normal bir akış olacağını söyledi; deneme farklı sonuç verdi. javac ile denendi: Görülen satır `Exception in thread "main" java.io.IOException: device error` oldu. Yani istisna `IOException`'dır, mesajı da görünür; "normal flow" ise yazılmaz.

12.4 throw ve throws Farkı

Özellik	`throw`	`throws`
Görevi	İstisnayı fırlatır	İstisnayı bildirir
Yanında ne yazılır?	Bir örnek, `new` ile üretilir	Sınıf adı
Nerede kullanılır?	Metot içinde	Metot imzasında
Birden çok istisna	Olmaz	Olur: `IOException` da `SQLException` da yazılabilir

Hoca `throws`'un checked istisnaları bildirmek için kullanıldığını da belirtti.

13. final, finally ve finalize

Hoca bu üçünün çok karıştırıldığını söyledi.

- **`final`:** Bir değişkenin değerini sabitler, `const` yapmak gibidir.
- **`finally`:** İstisna yönetimiyle ilgilidir. Kod hatalı çalışsın ya da çalışmasın, çalıştırılacak bölümü belirtir.
- **`finalize`:** Çöp toplayıcının (garbage collector) çalışmasıyla ilgili bir mekanizmadır.

Hocanın `finalize` örneğinde bir sınıftan `f1` ve `f2` nesneleri üretilir, sonra `null` yapılır ve çöp toplayıcı çalıştırılır. Metodu kimse doğrudan çağırmaz. Çöp toplayıcı nesneleri silerken çağırır.

Kod javac ile derlenip çalıştırıldı. İletinin ekrana düşmesini beklemek için `Thread.sleep` eklendi (ara adım eklendi).

```
public class FinalizeExample {
    protected void finalize() {
        System.out.println("finalize called");
    }
    public static void main(String[] args) throws Exception {
        FinalizeExample f1 = new FinalizeExample();
        FinalizeExample f2 = new FinalizeExample();
        f1 = null;
        f2 = null;
        System.gc();
        Thread.sleep(500);
    }
}
```

Çıktı:

```
finalize called
finalize called
```

> **Ek bilgi (derste anlatılmadı):** `finalize` artık önerilmez. JDK 25'te yansıma ile bakıldı: `Object.finalize` üzerinde `@Deprecated(forRemoval=true, since="9")` işareti var. Kaynak bırakmak için `AutoCloseable` ile `try-with-resources` kullanılabilir; bu biçim de javac ile denendi ve `close` metodu blok sonunda çağrıldı.

14. Override Edilen Metotlarda İstisna Kuralları

İstisnalar kalıtımla kullanıldığı için metot override edilirken uyulacak kurallar vardır. Hoca bunları ekranda derleyerek gösterdi. Aşağıda `Parent` üst sınıf, `Child` alt sınıftır. Sınıf adları ve iletiler anlatımdan kurulmuştur. Bütün durumlar javac ile denendi.

14.1 Üst Sınıf İstisna Bildirmiyorsa

Bu durumda alt sınıftaki metot **checked istisna bildiremez**, ama **unchecked istisna bildirebilir**.

```java
import java.io.IOException;

class Parent {
    void msg() { System.out.println("parent"); }
}

public class Child extends Parent {
    void msg() throws IOException {    // Derleme hatası
        System.out.println("child");
    }
    public static void main(String[] args) { }
}
```

Hata iletisi: `msg() in Child cannot override msg() in Parent` ve `overridden method does not throw IOException`.

> **Ders sonrası not:** Altyazıda hocanın `Exception` için "unchecked" dediği ve `IOException` için "checked olmadığı için" dediği yerler var (02:18:36-02:19:09); bu ifadeler büyük olasılıkla karışmış ya da yanlış çıkmıştır. Doğrusu: `Exception` ve `IOException` checked'tir. `throws Exception` da aynı hatayı verdi, javac ile denendi.

Hoca `IOException` yerine `RuntimeException` yazdı ve kod derlendi. Çalıştırılınca önce nesne oluşturulur, sonra metot çağrılır. Metot `RuntimeException` üretebilir diye bildirilmiştir, ama unchecked olduğu için `try-catch` içine alınmadı. Zaten bu örnekte üretmedi de.

Sonra aynı durumu gerçekten fırlatarak denedi. Yazarken `new`'i unuttuğunu fark etti: `throw new` denmelidir. Koşulsuz `throw`'dan sonraki satıra da derleyici itiraz etti:

```java
class Parent {
    void msg() { System.out.println("parent"); }
}

public class Child extends Parent {
    void msg() throws ArithmeticException {
        System.out.println("child");
        throw new ArithmeticException();
        System.out.println("rest of the msg");  // Derleme hatası
    }
    public static void main(String[] args) {
        new Child().msg();
    }
}
```

Hata iletisi `unreachable statement` oldu, çünkü buraya hiçbir zaman gelinemez. Hoca bir kaçış yolu gösterdi: `throw` satırı `if (1 == 1)` içine alınınca hata kalktı. Bu değişiklikle kod javac ile derlendi. Çalışınca `child` yazıldı, ardından `ArithmeticException` ile program bitti. `rest of the msg` yazılmadı.

14.2 Üst Sınıf İstisna Bildiriyorsa

Alt sınıftaki metot **aynı istisnayı, onun alt sınıfını ya da hiçbir istisna** bildirebilir. **Üst sınıfını bildiremez.** Yani aşağıdan yukarı çıkılamaz.

```java
class Parent {
    void msg() throws ArithmeticException {
        System.out.println("parent");
    }
}

public class Child extends Parent {
    void msg() throws Exception {      // Derleme hatası
        System.out.println("child");
    }
```

```
    public static void main(String[] args) { }
}
```

İki metot da `throws Exception` yapılınca kod derlenir. `Exception` checked olduğu için çağrı `try` içine alınır.

```
class Parent {
    void msg() throws Exception {
        System.out.println("parent");
    }
}

public class Child extends Parent {
    void msg() throws Exception {
        System.out.println("child");
    }
    public static void main(String[] args) {
        Parent p = new Child();
        try {
            p.msg();
        } catch (Exception e) {
            System.out.println(e);
        }
    }
}
```

Çıktı yalnız `child` olur, çünkü metot override edilmiştir. Üst sınıfa gidilmez.

14.3 super ile Üst Sınıf Metodunu Çağırmak

Soru: Üst sınıftaki `parent` iletisini de görmek için ne yapmalıyız?

Cevap: Bir öğrenci süper sınıftaki metodu çağırmayı önerdi. Hoca onayladı: Override metodu geçersiz kılar. İkisinin de çalışması isteniyorsa `super.msg()` yazılır.

`super.msg(); System.out.println("child");` sırasıyla çıktı `parent`, `child` olur. Bu sıra javac ile denendi.

Soru: `super.msg()` satırı `child` yazdıran satırdan sonra yazılabilir mi?

Cevap: Öğrenci yazılamayacağını söyledi. Hoca bunun constructor için geçerli olduğunu, burada böyle bir zorunluluk olmadığını açıkladı. Bu sıradan bir metot çağrısıdır: Önce `child`, sonra `parent` yazılır. Bu sıra da javac ile denendi ve aynı sonuç alındı.

> **Ders sonrası not:** Hoca constructor'da `super` çağrısının ilk satırda olması gerektiğini söyledi. Bu kural eski dil düzeyleri için geçerlidir; `-source 17` ile javac ile denendi ve `flexible constructors is not supported in -source 17` hatası verdi. JDK 25'in varsayılan düzeyinde ise `super(...)`'dan önce sıradan bir ifade yazmak derlendi ve çalıştı. Yani güncel sürümde bu kural değişmiştir.

Sonra hoca üst sınıfta `throws Exception`, alt sınıfta `throws ArithmeticException` yazıp `super.msg()` eklemeyi denedi. Derleyici uyarı verdi ve hoca süper'in burada kullanılamadığını söyledi. Bu satır kaldırılınca kod derlendi ve `child` yazıldı.

> **Ders sonrası not:** Hoca buradaki hatayı kesin bir kuralla açıklamadı. Sebep javac ile denenerek görüldü: `super.msg()` `Exception` fırlatabilir, alt sınıf metodu ise yalnız `ArithmeticException` bildiriyor. Hata iletisi `unreported exception Exception; must be caught or declared to be thrown` oldu.

Alt sınıf hiç istisna bildirmezse de kod derlenir; bu durum da javac ile denendi.

15. Özel İstisna Sınıfı Örneği ve System.err

Son örnekte hoca `InvalidAgeException` sınıfını yazdı. Sınıf `Exception`'dan türer. `throw` ile üretildiğinde bir mesaj gelir ve bu mesaj `super` ile üst sınıfa gönderilir. Hoca, kendi sınıfınızı

tasarladığınızda genellikle böyle bir constructor tanımlandığını söyledi. Uyarıları kapatmak için `@SuppressWarnings` kullanılabildiğini, bunun çok önemli olmadığını da ekledi.

```
class InvalidAgeException extends Exception {
    public InvalidAgeException(String s) { super(s); }
}

public class TestCustomException {
    static void validate(int age) throws InvalidAgeException {
        if (age < 18)
            throw new InvalidAgeException("not valid");
        System.out.println("welcome to vote");
    }
    public static void main(String[] args) {
        try {
            validate(13);
        } catch (InvalidAgeException e) {
            System.out.println(e.toString());
        }
        System.out.println("rest of the code");
    }
}
```

Çıktı:

```
InvalidAgeException: not valid
rest of the code
```

`not valid` metni `throw` satırından constructor'a gelir, oradan üst sınıfa gider. Hoca, `validate(13)` çağrısı `try` dışında bırakılınca `unreported exception` hatasını da gösterdi. Bu hata alınmasının iyi bir şey olduğunu, zaten beklendiğini söyledi.

> **Ders sonrası not:** Hoca "custom exception yazıyorsak `try-catch` zorunludur" dedi; bu eksiktir. Zorunluluk, sınıfın checked olmasından gelir. javac ile denendi: `RuntimeException`'dan türetilen özel istisna için `try` ve `throws` gerekmedi. `Exception`'dan türetilen özel istisnada da `try` yerine `main`'e `throws` yazmak derlemeyi geçti, ama program çalışırken istisnayla sonlandı.

15.1 System.err

Soru: `printStackTrace` çıktısı nerede görülüyor?

Cevap: Hoca çıktı penceresini gösterdi: Siyah olanlar `System.out.println` çıktısı, kırmızı olanlar hata çıktısıdır.

Hoca buna bir ekleme yaptı: `System.err.println("merhaba")` metni hata akışına gönderir. IDE'de kırmızı görünen yazılar aslında bu akışa yazılmış ifadelerdir. Akış kavramı dosyalar konusunda ayrıca görülecektir.

`printStackTrace` çıktısının `System.err`'e gittiği javac ile denendi: Standart hata akışı ayrı bir dosyaya yönlendirilince çağrı yığını ve `merhaba` o dosyaya düştü, `System.out` çıktısı düşmedi.

Dersin Toparlaması

Bu derste önce vize çözümleri üzerinden UML'e sadık kalmak, nesne kimliğini == ile kontrol etmek, arayüz türünde referans tutmak ve sekans diyagramında metot çağrılarını sırasıyla göstermek görüldü. Ardından hash fikri mod örneğiyle anlatıldı. `HashMap` ile benzersiz anahtara bağlı kayıtlar saklandı. Değeri liste olan `HashMap` ile basit bir veritabanı kuruldu. Collection Framework'te `Iterable`, `Collection`, `List`, `Queue`, `Set` hiyerarşisi, `Iterator`, `ArrayList` ve `LinkedList` incelendi.

İkinci yarıda istisna yönetimi anlatıldı. Kaçınma ile yönetim ayrıldı. `try`, `catch`, `finally`, `throw`, `throws` anahtar kelimeleri, checked ile unchecked ayrımı, özelden genele `catch` sırası, override kuralları ve özel istisna sınıfları görüldü.

Hoca, sonraki konuların dosyalar, iç sınıflar (inner class) ve thread’ler olduğunu söyledi. Nesneye yönelik programlamaya özgü kısımların hemen hemen burada bittiğini, dosya ve thread konularının nesneye yönelik olmayan dillerde de bulunduğunu belirtti. Üç dört slaytlık tip dönüştürme (typecasting) konusu da bir sonraki derse kaldı.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.